

# SSH Attack Simulation:

## Results of Different Defensive Techniques against Attack Scripts in a Closed Environment

Faizan Khan

CSEC 594

DePaul University

**Abstract**—This paper presents a project of a closed environment where a ssh port is targeted and attacked and the effectiveness of defensive controls are examined against a Cowrie honeypot. Using two virtual machines on an isolated VirtualBox network, three categories of simulated attack — credential spraying, post-authentication reconnaissance, and payload delivery were executed and captured. The implementation of three levels of conventional defensive controls, Fail2ban automated banning, UFW rate limiting, and iptables INPUT chain throttling had no measurable effect on attack traffic reaching the honeypot. There were difficulties experienced with the configuration of fail2ban onto cowrie which will be discussed. Analysis of the iptables packet counters revealed that Docker's network architecture bypasses the standard host firewall chains set up by UFW by editing iptables directly. A final defensive control implemented as an iptables rule in the DOCKER-USER chain produced complete attack blocking. The findings show that host-level firewall rules do not provide protection for services that are deployed within Docker unless the configuration is container-aware, which is significant for any organization to note if they are deploying services through containerization.

**Index Terms**—SSH security, honeypot, Cowrie, Docker security, iptables, firewall, Fail2ban, defensive controls, threat intelligence, deception technology.

### I. INTRODUCTION

Secure Shell (SSH) is one of the most universally deployed remote access protocols on the internet. It enables administrators to remotely access servers over an encrypted command line. Its widespread use has also made it one of the most targeted services. When an internet-facing SSH service is deployed, it is quickly identified by automated scanners, and the subsequent attack volume includes credential brute-force, distributed credential stuffing campaigns, post-authentication malware delivery, and attempts to use the compromised system as a launch point for additional attacks.

Despite widespread awareness of SSH-targeted threats, many organizations are still targets of SSH attacks, including automated scripted attacks.

It's important to note that this paper is a closed-laboratory investigation that addresses viability of a honeypot deployment and then evaluates the defensive controls against the attack patterns. The paper's primary contribution is the demonstration that conventional host-level firewall rules do not protect Docker-deployed services, a finding with broad implications beyond this closed environment honeypot project.

The remainder of this paper is organized as follows. Section II provides background on SSH attack patterns and honeypot technology. Section III details the methodology. Section IV describes the attack simulation scenarios. Sections V through VIII present results before and after each phase of defensive testing. Section IX discusses findings, Section X presents recommendations, Section XI addresses limitations, and Section XII concludes.

### II. BACKGROUND AND RELATED WORK

#### A. SSH Attack Taxonomy

The types of SSH attacks are distinct, and the volume, sophistication and intent of the attacks likewise vary. Brute forcing credentials is the process of trying every possible password against a list of known usernames, usually this is done in an automated fashion and at high volume. Credential stuffing is an attack where hackers use lists of stolen username/password combinations from past data breaches to take advantage of password reuse. Credential spraying attempts a few common passwords against many usernames to avoid lockout policies. Post-authentication reconnaissance is the enumeration of system information following a successful login. wget or curl to fetch malicious files for persistence payload delivery. No actual malicious payload was used for project.

#### B. Honeypot Technology

A honeypot is a decoy system that appears to be a real target but is used as an intelligence-gathering tool. The basic design principle is that any interaction with a honeypot is suspicious, providing zero false positive detection.

Honeypots can also be categorized by the level of interaction they provide. Low-interaction honeypots simulate the responses of services. High-interaction honeypots provide a real or close to real environment. Medium-interaction honeypots such as Cowrie provide a controlled middle ground with fake filesystems and shells that don't allow actual command execution [1].

### C. Existing Defensive Approaches

Common SSH defenses include hardening the daemon by editing `sshd_config`, using Fail2ban to automatically ban IPs based on log information, using UFW to make firewall management easier, and setting iptables rules directly for more granular control. Container-aware firewall rules are a less commonly discussed category that turned out to be central to the findings of this project.

## III. METHODOLOGY AND LAB SETUP

### A. Lab Environment

The laboratory consisted of two virtual machines on a VirtualBox host-only network, providing complete isolation from external networks [2]. VM 1 ran Ubuntu 22.04 LTS Server as the honeypot host (192.168.56.10); VM 2 ran Kali Linux as the simulated attacker (192.168.56.20). Both VMs were attached to the vboxnet0 host-only adapter with DHCP disabled. [2]

**TABLE I**  
LAB ENVIRONMENT CONFIGURATION

| VM   | Operating System | IP            |
|------|------------------|---------------|
| VM 1 | Ubuntu 22.04 LTS | 192.168.56.10 |
| VM 2 | Kali Linux       | 192.168.56.20 |

### B. Cowrie Honeypot Deployment

Cowrie was deployed with Docker with external port 22 mapped to Cowrie internal port 2222 and external port 23 to internal port 2223. Log and download directories were installed to the host file system for persistent access.

```
sudo docker run -d --name cowrie \ --restart \
unless-stopped \ -p 22:2222 -p 23:2223 \ -v \
/opt/cowrie/logs:... \ cowrie/cowrie:latest
```

### C. Attacker VM Setup

Kali Linux was imported as a pre-built VirtualBox appliance with a static IP. The installation of python libraries paramiko and sshpass was necessary for the three attack simulation scripts. ICMP echo was used to verify connectivity.

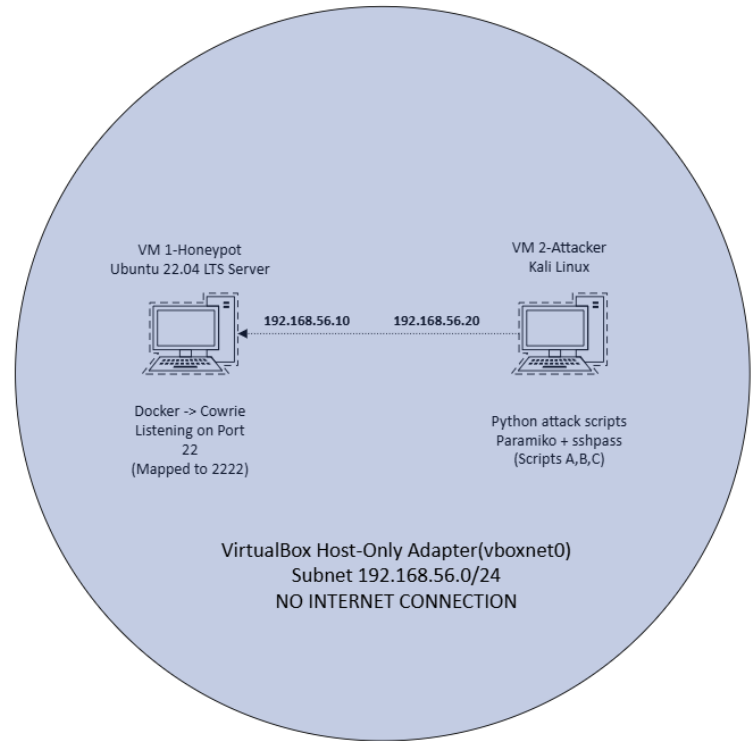


Fig. 1. Closed host-only network topology with VM-to-VM isolation.

## IV. ATTACK SIMULATION SCENARIOS

### A. Scenario A — Credential Spray

Twenty common username and password pairs were attempted in sequence with 0.5-second delays between attempts [3]. The delay mimics the throttled behavior of evasive attack tools known as low-and-slow attacks, which deliberately stay below standard rate-limiting thresholds.

### B. Scenario B — Post-Authentication Reconnaissance

Using paramiko [3], a successful authentication was followed by execution of eleven reconnaissance commands including `whoami`, `uname -a`, `cat /etc/passwd`, `ifconfig`, `netstat -an`, and `ps aux`[4]. These commands represent the typical enumeration pattern observed in published research on real-world SSH intrusions.

### C. Scenario C — Payload Delivery

A Python HTTP server on the attacker VM hosted a benign placeholder file [3]. The attack script issued `wget` and `curl` commands from the compromised honeypot to download the file, simulating malware delivery. No actual malware was used at any point.

## V. PHASE I: BASELINE RESULTS

The three scenarios were run multiple times before any defensive measures were taken. Using the logging interface provided by Docker, I pulled out the cowrie logs and got 310

lines of detailed attack telemetry [3]. “Successful logging of all connection attempts, credential pairs, post-authentication commands and download attempts of the payload.”

**TABLE II**  
BASELINE DATASET SUMMARY

| Data Captured          | Count                 |
|------------------------|-----------------------|
| Total log lines        | 310                   |
| Unique source IPs      | 1                     |
| Login attempts         | Multiple per scenario |
| Commands captured      | 10+ unique            |
| SSH client fingerprint | paramiko_4.0.0        |

Two notable findings emerged from the baseline. First, Cowrie successfully identified the attacking SSH client as paramiko 4.0.0, demonstrating useful tool fingerprinting capability of the honeypot. There were 10 attack sessions overall between the 3 scripts. Two pair of credentials that were consistently used were root./toor and root/password123.

## V. PHASE II: DEFENSIVE ATTEMPTS

### A. Fail2ban Configuration (Failed)

Attempts to install and configure Fail2ban were made [3]. Its purpose was to monitor Cowrie's logs and if it detected any brute force attempts, it would block the attack machine's IP address. A configuration typo initially prevented the jail from loading [5]. After correction, the regex validated successfully against existing logs but live integration with Docker's logging system proved fragile due to incompatibilities between Docker's internal logging architecture and Fail2ban's file watcher.

### B. UFW Rate Limiting

UFW was enabled with rate limiting on port 22, blocking IPs that exceed six attempts per thirty seconds.[3]

```
sudo ufw enable sudo ufw limit 22/tcp
```

### C. iptables INPUT Chain Throttling

Direct iptables rules were configured to drop connections from any IP exceeding five new connections per sixty seconds, using the recent module on the INPUT chain.[3].

## VII. PHASE III: DIAGNOSING THE FAILURE

### A. Observed Results

None of the three conventional controls produced any measurable effect on attack traffic reaching Cowrie. The problem was the docker that the honeypot was in was overriding any defenses. Table III summarizes the dataset across all phases.

**TABLE III**  
DEFENSIVE PHASE COMPARISON

| Dataset        | Lines | Effect   |
|----------------|-------|----------|
| Baseline       | 310   | None     |
| UFW rate limit | 351   | Bypassed |
| iptables INPUT | 354   | Bypassed |

### B. Root Cause Analysis

The UFW and iptables results required deeper investigation. Inspection of iptables packet counters revealed that the rate-limiting rules were correctly loaded but had matched zero packets and zero bytes throughout all attack runs:

```
Chain ufw-user-input pkts bytes target
source/dest 0 0 tcp dpt:22 ctstate
NEW
ufw... tcp dpt:22 ctstate NEW
recent: UPDATE seconds:30
hit_count:6
```

### C. Discovery — Docker Bypasses Host Firewall

Looking at the DOCKER iptables chain made it clear what was happening. When starting containers, Docker inserts its own iptables rules that redirect traffic for exposed container ports via a separate DOCKER chain that bypasses the regular INPUT chain [5]. Any host firewall rules configured to protect those ports are ignored silently.

```
Chain DOCKER ACCEPT tcp 0.0.0.0/0 172.17.0.2
tcp dpt:2222 ACCEPT tcp 0.0.0.0/0 172.17.0.2
tcp dpt:2223 DROP all 0.0.0.0/0 0.0.0.0/0
```

This bypass behavior was further verified through a control test: traffic to a Python HTTP server running directly on the host (without Docker) on port 8080 was correctly blocked by UFW, confirming that UFW itself was functioning. The Docker-bound SSH traffic on port 22 was simply never visible to UFW.

```
Chain DOCKER (1 references)
pkts bytes target prot opt in out source destination
0 0 ACCEPT tcp -- !docker0 docker0 0.0.0.0/0 172.17.0.2 tcp dpt:2223
0 0 ACCEPT tcp -- !docker0 docker0 0.0.0.0/0 172.17.0.2 tcp dpt:2222
0 0 DROP all -- !docker0 docker0 0.0.0.0/0 0.0.0.0/0
```

Fig. 2. Traffic flow through iptables chains. Docker-exposed ports bypass the INPUT chain entirely, routing through DOCKER chains that are not subject to user-configured INPUT rules.

## VIII. PHASE IV: THE WORKING DEFENSE

### A. The DOCKER-USER Chain

Docker provides a special iptables chain called DOCKER-USER for users to add their own firewall rules that interact with container traffic. This chain is checked before Docker's rules for routing traffic are checked, so user rules can reject access to a container. The DOCKER-USER chain is the correct place for container firewall rules, but interestingly it's rarely mentioned in manuals for Linux firewall configuration.

## B. Implementation

```
sudo iptables -I DOCKER-USER \ -s 192.168.56.20  
-j DROP sudo iptables -L DOCKER-USER -v -n
```

The rule inserts at the top of the DOCKER-USER chain (-I), matches packets from the attacker VM source IP (-s 192.168.56.20), and silently drops them (-j DROP).

## C. Result

Attack scripts on the attacker VM reported connection timeout errors. From the attacker's perspective, the honeypot ceased to respond. On the honeypot side, Cowrie logs contained no attack sessions whatsoever after the rule was applied — only startup messages and the SIGTERM from the container restart were present:

```
Cowrie Version 3.0.2 CowrieSSHFactory starting on  
2222 Ready to accept SSH connections [SIGTERM]  
Ready to accept SSH connections [no attack  
traffic logged]
```

DOCKER-USER rule has removed all attack-related log entries to zero from the baseline of 443 lines having around fifteen attack sessions. But all attempts to connect from attacker VM were silently dropped before reaching the container.

**TABLE IV**  
ALL DEFENSIVE PHASES SUMMARY

| Phase | Control              | Effect                |
|-------|----------------------|-----------------------|
| I     | None                 | All attacks logged    |
| IIa   | Fail2ban             | Integration failed    |
| IIb   | UFW                  | Docker bypass         |
| IIc   | iptables INPUT       | Docker bypass         |
| III   | Diagnosis            | Bypass identified     |
| IV    | iptables DOCKER-USER | <b>Complete block</b> |

## IX. DISCUSSION

### A. Docker Firewall Bypass Implications

The most interesting discovery of this academic project is that Docker ignores host firewall rules. However, organizations deploying any service via Docker, not just honeypots, may be under the false impression that their host firewall rules are in effect for those containers. If you're using UFW or configuring the iptables INPUT chain directly for basic rate limiting, IP blocking, or port restrictions, any ports exposed to Docker will just ignore these rules unless you set them up to work with Docker's network architecture.

### B. Honeypot vs. SSH Defense

Honeypots are deliberately exposed and are different from those used to protect real services. An effective measure for protecting ssh is ssh hardening. This can include changing the default port, disabling root login, enforce key-based

authentication, whitelisting exact users/groups that can access the SSH service, limit attempts per connection, etc [7].

## C. Fail2ban and Containerization

Fail2ban's integration with Cowrie's containerized logging is fragile, which is indicative of a larger operational concern [5]. Container logging architecture often does not like to cooperate with traditional log-watching tools, requiring custom log forwarding infrastructure or SIEM-based alternatives. The engineering required to make this work within the project's timeframe is difficult.

## X. DEFENSIVE RECOMMENDATIONS

Based on the findings, the following recommendations are presented in priority order:

- Audit Docker firewall configurations. Verify that host firewall rules apply to containerized services. Insert restrictions into the DOCKER-USER chain rather than the INPUT chain.
- Disable password authentication entirely. With key-based authentication enforced, credential stuffing becomes structurally impossible.
- Restrict root login. Root is the most-targeted username; removing it from the attack surface forces attackers to first compromise an unprivileged account.
- Move SSH off port 22. This reduces noise from automated scanners and improves the signal-to-noise ratio of remaining attack traffic.
- Deploy Fail2ban with aggressive thresholds, but only on non-containerized SSH services. Engineering a method for fail2ban's file watcher to view cowrie's logs within the docker container will take considerable effort.
- You may deploy honeypots on unused internal IPs as tripwires. Any traffic to a honeypot is malicious, providing zero-false-positive detection.
- Defense In Depth. No single control is sufficient; effective architecture combines authentication hardening, container-aware firewall rules, automated banning, and deception-based monitoring.

## XI. LIMITATIONS

The main limitation of the project is that all attack traffic was simulated by scripts controlled by the researcher. Simulated attackers do not improvise nor have the full unpredictability of real adversaries. Future work would extend the methodology to internet-exposed deployments. Further, only five defensive controls were tested; threat-intelligence blocklists, GeoIP filtering, and multifactor authentication implementations were out of scope. Docker firewall bypass results are specific to Docker's default networking mode and may not be portable to other container runtimes such as Podman or LXC.

## **XII. CONCLUSION**

This project investigated SSH targeted attacks through controlled empirical tests in a closed laboratory setting. Three simulated attack scenarios were captured by a Cowrie honeypot and four defensive controls were tested in sequence. With the failure of three conventional controls leading to no measurable effects, it did lead to an architectural discovery that explained all of them: Docker's network architecture silently bypasses standard host firewall rules. A single rule in Docker's dedicated DOCKER-USER chain completely blocked attack traffic.

The story arc of baseline to three failed defenses to a working solution is the reality of security operations work. Defenses fail more often than tutorials suggest and diagnostic skill is itself a trail of skill. The most actionable takeaway beyond the technical findings is that organizations that depend on host-level firewall rules to secure services deployed by Docker may have significantly less protection than they believe. Effective SSH defense requires layered defenses that combines authentication-layer hardening, container-aware firewall configuration, and deception-based detection.

## **. REFERENCES**

- [1] M. Oosterhof, "Cowrie SSH/Telnet Honeypot," GitHub repository. [Online]. Available: <https://github.com/cowrie/cowrie>
- [2] Oracle Corporation, VirtualBox User Manual, 2024. [Online]. Available: <https://www.virtualbox.org/manual/>
- [3] F.Khan"CSEC-594-Capstone\_Files," GitHub repository. [Online]. Available: [https://github.com/faizdagreat/CSEC-594-Capstone\\_Files](https://github.com/faizdagreat/CSEC-594-Capstone_Files)
- [4] J. Forcier, "Paramiko: Python implementation of SSHv2," Paramiko Project. [Online]. Available: <https://www.paramiko.org/>
- [5] "Fail2ban Documentation," Fail2ban Project. [Online]. Available: <https://www.fail2ban.org/>
- [6] "Docker and iptables," Docker Documentation. [Online]. Available: <https://docs.docker.com/network/iptables/>
- [7] J. Forshaw, OpenSSH sshd\_config Reference Manual, OpenBSD Project. [Online]. Available: [https://man.openbsd.org/sshd\\_config](https://man.openbsd.org/sshd_config)